E-Commerce Product Entity-Resolution & Matching Platform

Team ID: 29 | Section: 10 | Members: K. Sathwika – 2520090210, A. Sanjana – 2520090159 | Guide: Ms. K. Chandusha

Data Structures and Algorithms - 3 (25CS2103E) | KLH (Deemed to be University), Bachupally Campus | A.Y. 2026-2027

Abstract

- A Java-based entity-resolution engine that identifies identical products listed under different titles, sellers and formats and merges them into one canonical record. Hand-built data structures (hash maps, heaps, union-find, graphs) drive every step.
- Blocking, similarity graphs and clustering resolve large catalogs into golden records, delivering a de-duplicated catalog and cross-seller price comparison.

Problem Statement & Objectives

- **Problem:** The same product is listed thousands of times with inconsistent titles, units and specs. Manual de-duplication is slow, does not scale, and fragments reviews, ratings and prices across duplicate listings.
- **Objective 1:** Automate the matching of identical products listed under different titles, sellers and formats.
- **Objective 2:** Normalize attributes (brand, size, model) into a unified taxonomy and score candidate pairs with a transparent weighted score.
- **Objective 3:** Use blocking, similarity graphs and union-find clustering to resolve catalog entries into golden records without a full $O(n^2)$ comparison.
- **Scope:** Multi-seller listings through to canonical product records with cross-seller price comparison.

Existing vs Proposed System

Feature	Existing System	Proposed System
Matching approach	Rule-based SKU/UPC exact match; fails on mistyped or missing barcodes	Multi-signal weighted score: attribute + text + image similarity
Variant handling	TF-IDF / cosine on titles; confuses similar colors and sizes	Brand, model, size and unit normalized to a canonical taxonomy
Scale	Manual de-duplication; slow and unscalable	Hash-based blocking prunes candidate pairs; union-find clusters
Output & audit	Fragmented reviews, ratings and prices	Golden records with price comparison; every merge explainable

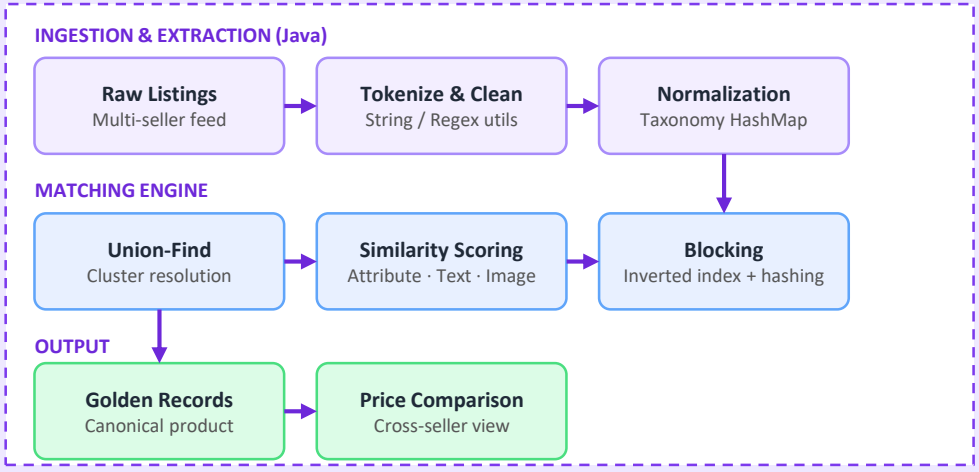
Hardware Requirements & Scalability

- **Processor:** Multi-core CPU (Intel i5/i7 or AMD Ryzen 5+) for parallel similarity scoring.
- **Memory:** 16 GB RAM minimum (32 GB recommended) to hold catalog-scale similarity graphs.
- **Storage:** 512 GB SSD for fast I/O across inverted-index and cluster-graph lookups.
- **Display:** Full HD (1920×1080) for dashboard and interface testing.
- **Scalability:** Can be hosted on AWS or Azure to scale compute during peak catalog-ingestion and flash-sale periods.

Tools & Technologies

- **Language:** Java 17+ with the JDK Collections Framework for the engine logic
- **Text Processing:** Apache Lucene / OpenNLP; Apache Commons Text (Jaro-Winkler, Levenshtein)
- **Core DSA (hand-built):** HashMap, PriorityQueue, DisjointSet, Inverted Index, Graph
- **Database:** PostgreSQL or MongoDB via JDBC / MongoDB Java Driver
- **Build & Test:** Maven / Gradle, JUnit 5; IDE: IntelliJ IDEA / Eclipse

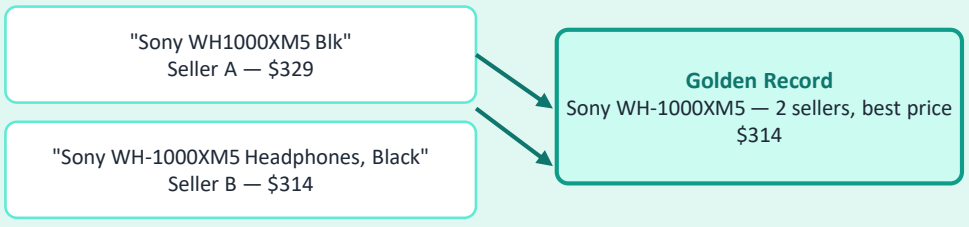
System Architecture



Core DSA Concepts

DSA Concept	Java Realization	Used For	Complexity
Hash Map	Custom HashMap<K,V> (chaining)	Inverted index & attribute lookup	O(1) avg
Priority Queue	Custom MinHeap / PriorityQueue<T>	Live ranking of match candidates	O(log n)
Disjoint Set	Custom DisjointSet (union-find)	Clustering listings into one product	$O(\alpha(n)) \approx O(1)$
Graph (Adj. List)	HashMap<Integer, List<Integer>>	Cross-seller similarity network	O(V + E)
Trie / Inverted Index	Custom Trie or token-postings map	Prefix & substring title search	O(k), k = key length
Sorting	Merge Sort on Comparable<Product>	Deterministic candidate ordering	O(n log n)

Product Matching Example



Each node is a listing; BFS/DFS connected components collapse every listing of the same product into one canonical record.

Results & Performance



76% faster time-to-match vs. manual review; 0 duplicate golden records after clustering.

Conclusion

- Built a Java engine on hand-implemented data structures that resolves multi-seller listings into canonical golden records.
- Meets all objectives: automated matching, explainable resolution and scalable clustering, with cross-seller price comparison.

Future Scope

- **Price Intelligence:** track price movement across clusters to flag anomalies and predict fair pricing.
- **Counterfeit Detection:** flag listings that mimic a golden record without trusted seller signals.
- **Multilingual Catalogs:** extend normalization to non-English titles and regional formats.

References: [1] T. H. Cormen et al., Introduction to Algorithms, MIT Press [2] P. Christen, Data Matching, Springer, 2012 [3] Apache Lucene Documentation, lucene.apache.org [4] Apache Commons Text Documentation, commons.apache.org/proper/commons-text [5] Java SE 17 Documentation, docs.oracle.com/en/java/javase/17

Github QR : https://github.com/sathwikakodam2007-hue/DSA-3-Project